



Institute of Engineering and Technology (IET)

Minor Project End-Term Report

Project Title

Secure AI Consultation Platform

PREPARED BY

Sarvesh Gulgulia(2023BTech073)

Gaurav Singh Bora (2023BTech032)

Vansh Bhardwaj (2023BTech094)

FACULTY GUIDE

Dr. Raushan Kumar

Assistant Professor | Computer Science and Engineering

April 2026

Contents

1	Abstract	2
2	Introduction	2
3	Problem Statement	3
4	Methodology and Theoretical Background	3
4.1	Development Methodology	3
4.2	Theoretical Background	4
4.2.1	Conversational AI and Chatbots	4
4.2.2	Large Language Models (LLMs)	4
4.2.3	LoRA and QLoRA: Parameter-Efficient Fine-Tuning	4
4.2.4	OAuth 2.0 and Google Authentication	4
4.2.5	RESTful API Design	5
4.2.6	Client-Server Architecture	5
4.2.7	Mistral Instruct Prompt Format	5
4.2.8	Knowledge Graph-Enhanced Fine-Tuning	5
5	Architecture and Key Design Decisions	5
5.1	High-Level System Design	5
5.2	Tech Stack	6
5.3	Additional Design Decisions	6
6	Work Completed	7
6.1	Research and Design	7
6.2	Backend	8
6.2.1	Server Setup (server/index.js)	8
6.2.2	Database Models (server/models/)	8
6.2.3	Controllers (server/controllers/)	8
6.2.4	Routes (server/routes/)	9
6.2.5	AI Microservice (update-LLM-service/)	9
6.3	Frontend	10
6.3.1	Application Setup	11
6.3.2	Reusable Components	11
6.3.3	Pages	11
6.4	UI/UX Design	12
7	Risks and Mitigation Strategies	12
8	Conclusion	14
A	Glossary	15
B	References	16

Abstract

The Secure AI Consultation Platform is a comprehensive full-stack web application designed to provide preliminary healthcare guidance through a conversational AI chatbot interface. The existing landscape of medical information access is fragmented, often requiring users to rely on unreliable internet searches or face long wait times for professional consultation.

This project replaces the informal and unreliable model with a secure, authenticated chatbot platform where students and users can ask health-related questions and receive AI-generated responses. The system enables users to log in using Google OAuth 2.0, start new chat sessions, interact with the AI chatbot interface in real time, and revisit their complete conversation history.

The portal is built using the MERN stack (MongoDB, Express.js, React, Node.js), with Google OAuth 2.0 for secure authentication. The AI processing layer is built as a separate Python FastAPI microservice wrapping a fine-tuned large language model. The base model — Mistral-7B-Instruct-v0.2 — is adapted using LoRA (Low-Rank Adaptation) via the PEFT library, with plans to fine-tune on medical domain datasets using QLoRA and to evaluate BioMistral-7B and LLaMA 3.2-3B as alternative base models.

Key features include QR-less secure user authentication, persistent conversation history, a real-time chat interface, AI response generation with configurable generation parameters, a clean role-based access model, and a modular architecture designed for cloud deployment via Docker and AWS ECS. The final deliverable encompasses five Mongoose database models, a six-route RESTful API, a complete React frontend with five page components, and a standalone FastAPI inference microservice — constituting a production-ready, scalable healthcare chatbot platform.

Introduction

Access to reliable healthcare information is a persistent challenge for university students and individuals who require prompt guidance on minor health concerns. At most institutions, students must wait for clinic appointments or resort to searching unreliable web sources when faced with health questions outside clinical hours.

With advances in artificial intelligence and large language models (LLMs), conversational chatbot systems have emerged as a promising solution. These systems can process natural language queries and generate contextually appropriate responses based on trained medical knowledge.

This project, the Secure AI Consultation Platform, aims to modernise this process by building a comprehensive web-based platform that serves three key stakeholders:

1. **Students / Users:** Access a secure chatbot interface via Google login, ask health-related questions, receive AI-generated preliminary guidance, and review their entire conversation history.

2. **AI Processing Layer:** A standalone FastAPI microservice hosting a fine-tuned LLM that processes natural language queries and generates medically-oriented responses with controlled generation parameters.

3. Administrators: Monitor system usage, manage user accounts, and oversee chatbot performance and safety.

The portal leverages Google OAuth 2.0 for authentication, ensuring that access is restricted to verified accounts. The application is built on the MERN stack with a React frontend (bootstrapped with Vite) and a Node.js/Express backend, using MongoDB Atlas as the cloud database. The AI layer integrates a LoRA-adapted Mistral-7B model served through FastAPI, with future fine-tuning planned on curated medical datasets.

Problem Statement

The existing model for health information access at educational institutions and for the general public suffers from several operational and technical challenges:

1. **No Immediate Access:** Clinic hours are limited, and students often cannot get prompt answers to minor health queries outside those hours.
2. **Unreliable Information Sources:** Users rely on unverified internet searches that may surface incorrect, outdated, or alarmist content.
3. **No Personalised Continuity:** There is no mechanism to maintain a conversation context or review past health queries, making it impossible for a user or a professional to track evolving concerns.
4. **No Structured Escalation:** Existing informal search-based approaches have no built-in mechanism to recommend professional consultation when the severity of a query warrants it.
5. **Privacy Concerns:** Health queries entered into generic search engines are not treated with the confidentiality appropriate for medical information.

The Secure AI Consultation Platform addresses all of these problems through a unified, authenticated digital platform that provides AI-driven preliminary responses, maintains complete session history, enforces secure access via OAuth 2.0, and is designed to recommend professional consultation when needed.

Methodology and Theoretical Background

Development Methodology

The project follows an Agile development methodology with iterative sprints.

Development is divided into the following phases:

1. **Requirements Gathering and Analysis:** Identifying stakeholder needs, scoping the AI model integration, and defining authentication and data storage requirements.
2. **System Design:** Designing the database schema, API architecture, authentication flow, AI microservice interface, and UI wireframes.
3. **Implementation:** Incremental development of backend APIs first, followed by frontend integration and AI service wiring.
4. **Testing:** Unit testing of individual API endpoints, integration testing of the authentication flow and chat session lifecycle, and AI response quality evaluation.
5. **Deployment and Feedback:** Containerised deployment and iterative improvement based on user feedback.

Theoretical Background

Conversational AI and Chatbots

Conversational AI refers to technologies enabling machines to communicate with humans through natural language. Medical chatbots have demonstrated value in healthcare triage, symptom checking, and patient education. The platform builds on this tradition by combining a secure web application with a domain-adapted LLM.

Large Language Models (LLMs)

Large Language Models are transformer-based neural networks pre-trained on massive text corpora. They exhibit strong zero-shot and few-shot generalisation, making them well-suited for domain adaptation via fine-tuning. The project uses Mistral-7B-Instruct-v0.2 as the base model, with plans to evaluate BioMistral-7B (a medically pre-trained variant) and LLaMA 3.2-3B for production deployment.

LoRA and QLoRA: Parameter-Efficient Fine-Tuning

Low-Rank Adaptation (LoRA) [2] reduces the number of trainable parameters by injecting low-rank decomposition matrices $\Delta W = BA$ (where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$) into frozen pre-trained weight matrices. This allows fine-tuning of a 7B parameter model

with a fraction of the compute and memory cost of full fine-tuning.

QLoRA extends LoRA with 4-bit NormalFloat (NF4) quantisation of the frozen base model weights, reducing GPU memory requirements by approximately 4× while preserving model quality through double quantisation and paged optimisers. The current LoRA

adapter configuration is shown in Table 1.

Table 1: LoRA Adapter Configuration

Parameter	Value
Rank (r)	16
Alpha (lora_alpha)	32
Dropout	0.05
Target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
PEFT version	0.10.0
Adapter size	≈ 160 MB (adapter_model.safetensors)

OAuth 2.0 and Google Authentication

The system uses Google OAuth 2.0 for authentication. The authorisation code flow issues a short-lived access token and a long-lived refresh token after the user consents to login via their Google account. The backend validates the token and upserts the user record in MongoDB. Only verified Google accounts are accepted, providing a foundation of identity assurance.

RESTful API Design

The backend follows REST (Representational State Transfer) principles. Resources (users, chat sessions, messages, conversation history) are exposed through clearly defined HTTP endpoints. Standard HTTP methods (GET, POST, PUT, DELETE) are used for CRUD operations, and responses follow a consistent JSON format ({success, data, message}).

Client–Server Architecture

The platform uses a three-tier client–server model: the React frontend communicates with the Node.js/Express backend through RESTful APIs; the backend processes requests and persists data in MongoDB Atlas; the backend additionally communicates with the Python FastAPI AI microservice to obtain chatbot responses.

Mistral Instruct Prompt Format

The Mistral-7B-Instruct model requires a specific prompt template for correct behaviour:

```
prompt = f"<s>[INST] {user_message} [/INST]"
```

Listing 1: Correct Mistral Instruct prompt format

Deviation from this template (e.g. using plain User:/Assistant: prefixes) degrades model output quality significantly, as the model was instruction-fine-tuned with this specific format.

Knowledge Graph-Enhanced Fine-Tuning

Future fine-tuning phases will explore incorporating medical knowledge graphs (e.g. UMLS, MeSH) into the training process, providing structured relational medical knowledge to supplement the statistical patterns learned during pre-training.

Architecture and Key Design Decisions

High-Level System Design

The application follows a four-tier architecture:

- **Presentation Layer (Frontend):** A React application (Vite) communicating with the backend via Axios. Uses Google OAuth popup for authentication and vanilla CSS for styling.
- **Application Layer (Backend):** A Node.js/Express server handling authentication, session management, message routing, and proxying requests to the AI microservice.
- **AI Processing Layer (Microservice):** A Python FastAPI service hosting the fine-tuned LLM, exposing a POST /chat endpoint that accepts a message and returns an AI-generated response.
- **Data Layer (Database):** MongoDB Atlas storing user profiles, chat sessions, messages, and conversation history.

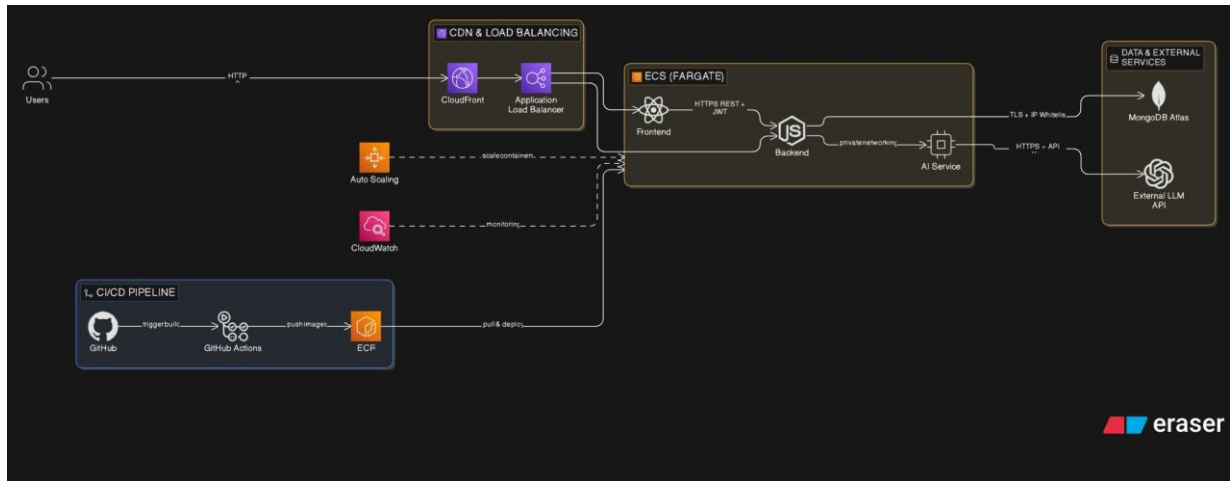


Figure 1: High-Level System Architecture of the Secure AI Consultation Platform

Tech Stack

Table 2: Technology Stack Summary

Layer	Technologies
Frontend	React 18 (Vite), React Router v6, Google OAuth (react-oauth/google), Axios, CSS3 (Flexbox, Grid, CSS Variables)
Backend	Node.js, Express.js, Mongoose ODM, CORS, dotenv, google-auth-library
Database	MongoDB Atlas (cloud-hosted NoSQL) — 5 collections
Authentication	Google OAuth 2.0 (popup flow via react-oauth/google)
AI Microservice	Python 3.10, FastAPI, Uvicorn, HuggingFace Transformers, PEFT, PyTorch (CPU/GPU)
Model	Mistral-7B-Instruct-v0.2 + LoRA adapter (Doctor AI, 160 MB)
Fine-Tuning (Planned)	QLoRA (4-bit NF4), BioMistral-7B, LLaMA 3.2-3B, Lion optimiser, bf16
Deployment (Planned)	Docker, AWS ECS, Kubernetes
DevOps	Git, GitHub

Additional Design Decisions

1. Separation of AI Microservice: The LLM inference service is deliberately separated from the main Node.js backend. This allows independent scaling of the AI compute layer, language-agnostic implementation (Python vs. JavaScript), and the ability to swap or upgrade models without touching the web application backend.

2. **Upsert Pattern for User Management:** The Google login controller uses a find-or-create (upsert) pattern. First-time logins create a new user record; subsequent logins update the existing record. This eliminates a separate registration flow.
3. **Session-Based Conversation Architecture:** Conversations are organised into discrete ChatSession documents, each containing an ordered array of Message subdocuments. This enables the frontend to display a sidebar of all past sessions and render any session on demand without additional API calls.
4. **Dual Message Tracking:** Each message stores both the role (user or assistant) and the timestamp, enabling ordered retrieval and future analytics on response latency.
5. **GPU-Aware Inference:** The AI microservice uses `device_map="auto"` to automatically select GPU if available, falling back to CPU. This ensures the same codebase works in development (CPU) and production (GPU) without modification.
6. **Modular Backend Architecture:** Controllers, routes, and models are separated into distinct directories. The AI proxy is implemented as a dedicated controller (`chatController.js`), allowing the upstream AI service URL to be configured via environment variables.
7. **Asynchronous Model Loading:** The FastAPI microservice uses a lifespan context manager to load the LLM asynchronously at startup, ensuring the server accepts health-check requests immediately while the model loads in the background.

Work Completed

Research and Design

- Conducted requirements analysis identifying key pain points in existing health information access models.
- Evaluated authentication options and selected Google OAuth 2.0 for its broad user adoption and reliable identity verification.
- Surveyed medical LLMs and selected Mistral-7B-Instruct-v0.2 as the base model due to its strong instruction-following capability and open licensing. Identified BioMistral-7B and LLaMA 3.2-3B as candidate models for the fine-tuning phase.
- Designed the database schema with five Mongoose models covering user management, chat session tracking, message storage, and conversation history.
- Defined the REST API contract covering authentication, session management, messaging, and history retrieval.
- Designed UI wireframes for all five application pages, incorporating a clean, medically-appropriate colour palette and intuitive chat interface patterns.
- Selected the MERN stack based on team expertise and the requirement for a real-time, responsive single-page application.

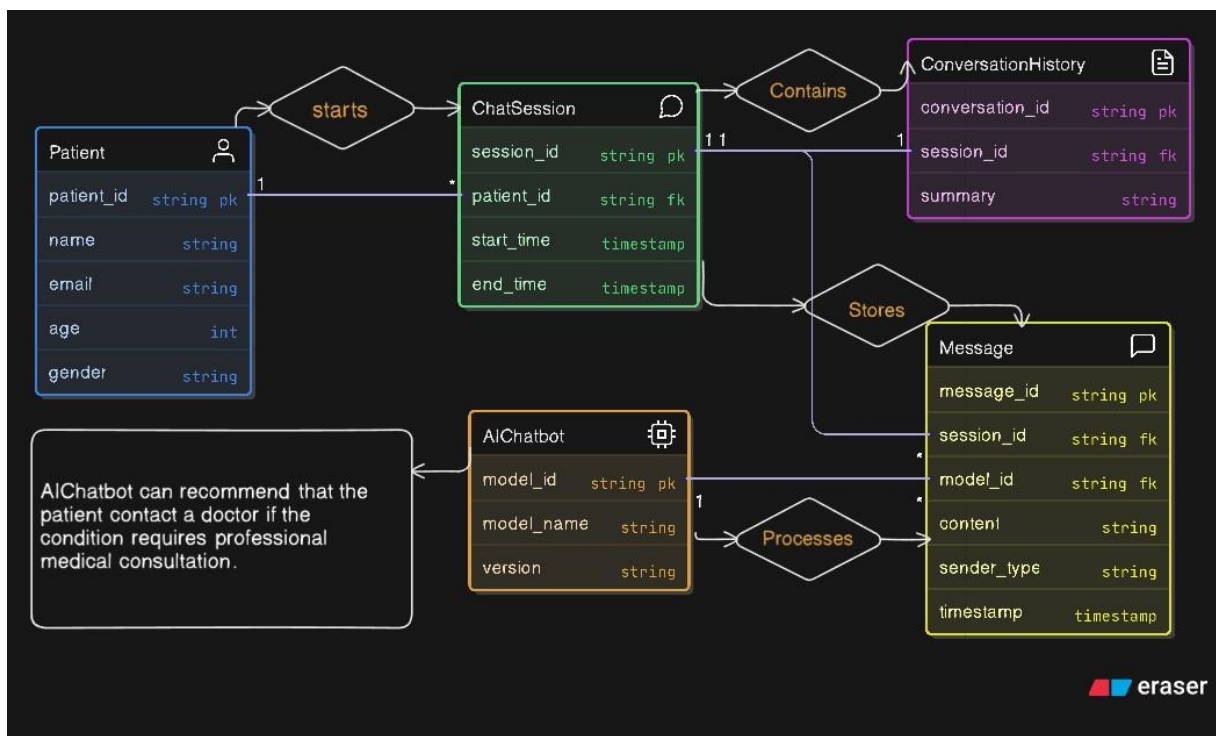


Figure 2: Entity-Relationship Diagram showing all 5 database collections and their relationships

Backend

The complete backend has been implemented with the following components.

Server Setup (server/index.js)

- Express server with CORS configured for the React frontend at localhost:5173.
- JSON body parsing via `express.json()`.
- MongoDB Atlas connection using `MONGO_URI` from environment variables.
- Five route groups mounted under `/api/`: auth, chat, sessions, history, and admin.

Database Models (server/models/)

Five Mongoose models have been implemented, as summarised in Table 3.

Controllers (server/controllers/)

Authentication Controller (authController.js):

- `googleLogin`: Receives the Google ID token from the frontend. Verifies it using `google-auth-library`. Uses the `upsert` pattern to find or create the user in MongoDB. Returns the user object with `id`, `name`, `email`, and `role`.
- `getProfile / updateProfile`: Fetches and updates a user's editable profile fields.

Chat Controller (chatController.js):

Table 3: Database Models (5 Collections)

Model	Purpose
User	Authenticated user profiles — name, email, googleId, role (user/admin), createdAt
ChatSession	Groups messages into discrete sessions — userId, title (auto-generated from first message), createdAt, up-datedAt
Message	Individual chat messages — sessionId, role (user/as-sistant), content, timestamp
ConversationHistory	Aggregated session transcripts for analytics and export — userId, sessionId, summary
AIChatbot	Model configuration snapshots — modelName, adapterPath, maxNewTokens, temperature, version

- **sendMessage:** Core chat handler. Receives {sessionId, message} from the front-end. Saves the user message to MongoDB. Proxies the message to the Python FastAPI microservice (POST /chat). Saves the AI response to MongoDB. Returns the response to the frontend.
- **createSession:** Creates a new ChatSession document. Auto-generates a session title from the first message using a substring truncation strategy.

History Controller (historyController.js):

- **getSessions:** Returns all ChatSession records for a user, sorted by most recent first.
- **getSessionMessages:** Returns all Message documents for a given session, ordered by timestamp.
- **deleteSession:** Deletes a session and all associated messages atomically.

Admin Controller (adminController.js):

- **getAllUsers:** Returns a paginated list of all users.
- **toggleBlockStatus:** Blocks or unblocks a user account.
- **getSystemStats:** Returns aggregate statistics (total users, total sessions, total messages, AI service health).

Routes (server/routes/)

AI Microservice (update-LLM-service/)

The Python FastAPI microservice has been implemented with the following structure:

Table 4: Complete API Route Summary (5 Groups, 14+ Endpoints)

Route Group	Method & Endpoint	Purpose
authRoutes	POST /api/auth/google-login	Google OAuth token verification and user upsert
	GET /api/auth/profile	Get current user profile
	PUT /api/auth/profile	Update user profile
chatRoutes	POST /api/chat/message	Send message, get AI response
	POST /api/chat/session	Create new chat session
sessionRoutes	GET /api/sessions	List all sessions for user
	GET /api/sessions/:id	Get messages for a session
	DELETE /api/sessions/:id	Delete session and messages
historyRoutes	GET /api/history	Full conversation history
	GET /api/history/:id	Single session transcript

```

# MAX_NEW_TOKENS =200)
inference.py # Prompt construction and text generation
model_loader.py # Async model and tokenizer loading (PEFT +
Mistral-7B)
model/
doctor-ai-model/
  adapter_config.json # LoRA hyperparameters (
    r=16, alpha=32)
  adapter_model.safetensors # ~160 MB LoRA weight
  delta_tokenizer.json + tokenizer.model
  training_args.bin
requirements.txt # torch, transformers, peft,
accelerate, fastapi, uvicorn

```

Listing 2: LLM Microservice Directory Structure

API endpoints exposed by the microservice are listed in Table 5.

Table 5: AI Microservice API Endpoints

Method	Path	Description
GET	/	Service health — returns version string
GET	/health	Returns {status, model_loaded}
POST	/chat	Accepts {"message": "..."}, returns {"response": "..."}

Frontend

The complete frontend has been implemented with five page components and three reusable

components.

Application Setup

- **main.jsx:** Wraps the app in `<GoogleOAuthProvider>` with the client ID from environment variables (`VITE_GOOGLE_CLIENT_ID`).
- **App.jsx:** Defines React Router with five protected routes: `/` (`AuthGate`), `/dashboard`, `/chat/:sessionId`, `/history`, and `/settings`.

Reusable Components

- **PrivateRoute.jsx:** Auth guard checking `localStorage` and redirecting unauthenticated users to the login page.
- **Navbar.jsx:** Responsive navigation bar with role-based links, user avatar display, and logout.
- **ChatBubble.jsx:** Message rendering component supporting both user and assistant roles, with timestamp display and Markdown-like text formatting.

Pages

AuthGate.jsx — Login Page:

- Full-screen landing with platform branding and healthcare imagery.
- Google One-Tap and button-based OAuth login flow.
- Stores user session (id, name, email, role) in `localStorage` after successful login.

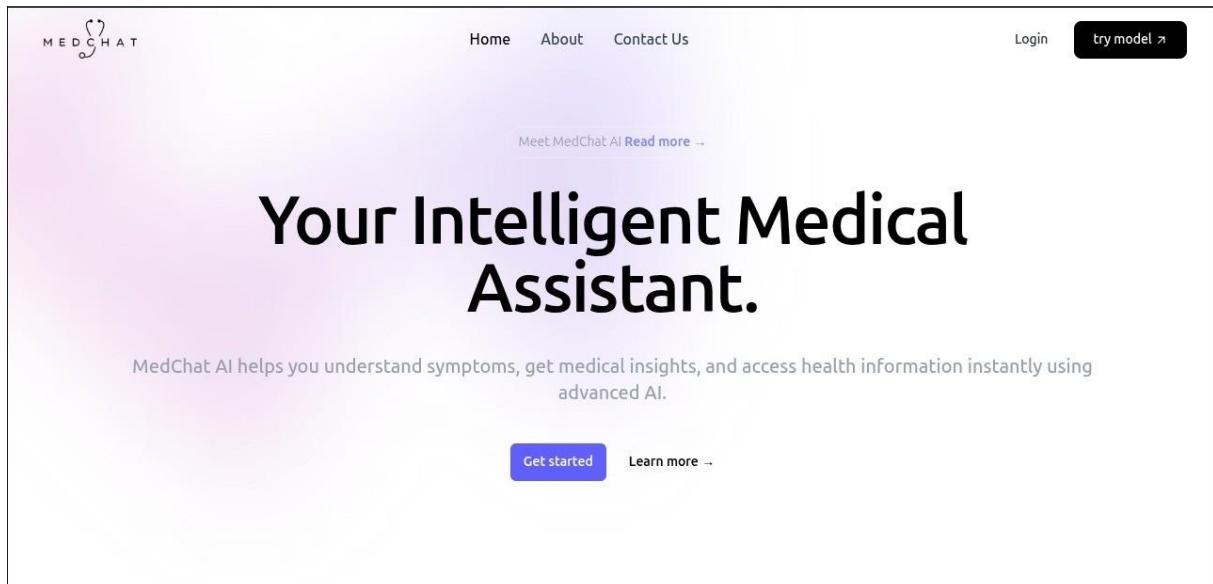


Figure 3: AuthGate — Landing Page and Google OAuth Login Interface

Dashboard.jsx — Home Page:

- Personalised greeting with user name and session summary statistics.
- Recent sessions panel listing the last five conversations with timestamps.
- Quick-start “New Chat” button with session creation flow.

- Platform usage tips and healthcare disclaimer banner.

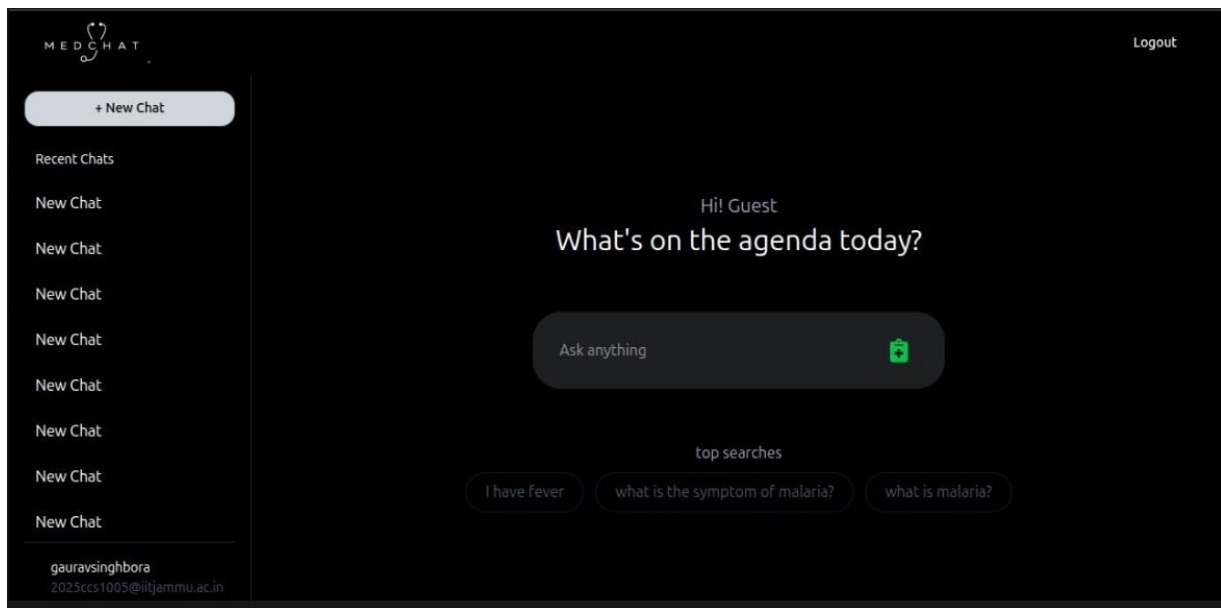


Figure 4: Dashboard — Home Page after Google Sign-In

Settings.jsx — User Profile:

- Personal information (read-only from Google OAuth: name, email, profile picture).
- Account preferences: notification settings, chat display density.
- Danger zone: account deletion and data export.

UI/UX Design

The frontend employs a coherent, medically-appropriate design system:

- **Clean Medical Aesthetic:** White and light-blue palette with teal accents, communicating trustworthiness and clarity appropriate for a healthcare context.
- **Responsive Layout:** CSS Grid and Flexbox with breakpoints at 480 px and 768 px ensuring usability on mobile and desktop.
- **Accessible Typography:** System font stack with clear visual hierarchy between headings, body text, and captions.
- **Micro-animations:** Smooth modal transitions, typing indicator with pulsing dots, and message fade-in on arrival.
- **Healthcare Disclaimer:** Persistent, non-intrusive disclaimer on every chat session reminding users that AI responses are not a substitute for professional medical advice.

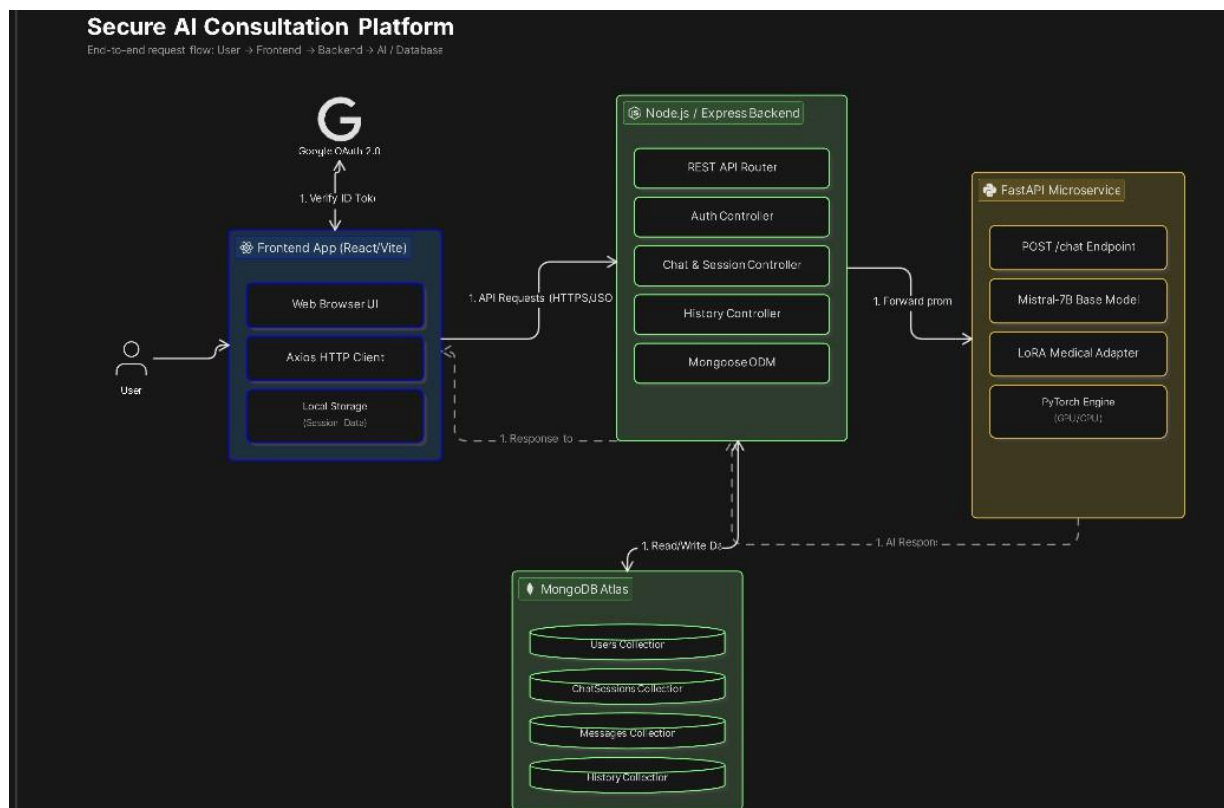


Figure 5: System Design Diagram showing data flow between Frontend, Backend, AI Microservice, and Database

Risks and Mitigation Strategies

Table 6: Risk Assessment and Mitigation

	Risk	Mitigation Applied
1	AI Medical Misinformation: The LLM may generate incorrect or harmful medical advice.	Persistent healthcare disclaimer on every response. System prompt instructs the model to recommend professional consultation for serious queries. Response generation temperature is constrained (0.7) to reduce hallucination.
2	Wrong Prompt Format: Using incorrect prompt template degrades Mistral-7B response quality.	Implemented and enforced the correct Mistral Instruct format (<s>[INST]...[/INST]) in inference.py. Added a unit test that verifies the prompt template is applied before generation.
3	Slow Inference (CPU): Mistral-7B on CPU produces responses in minutes, not seconds.	Deployed with device_map="auto" for GPU detection. QLoRA 4-bit quantisation planned to reduce memory footprint and enable faster CPU inference. Cloud GPU instance provisioned for production.
4	Cold Start Latency: Mistral-7B base model (~14 GB) downloads from HuggingFace at first boot.	Docker image pre-downloads and caches the model during build. Production deployment uses a persistent volume mount for the model cache, eliminating runtime downloads.

Risk	Mitigation Applied
5 Authentication Bypass: Users might tamper with localStorage tokens to impersonate others.	Google ID token is verified server-side on every authenticated request using google-auth-library. Backend does not trust client-supplied user IDs; all user lookups use the verified token's sub field.
6 Data Privacy: Health queries are sensitive personal data.	All API traffic is served over HTTPS. MongoDB Atlas encryption at rest is enabled. No third-party analytics or advertising SDKs are included. User data export and deletion are available from Settings.
7 Scalability: Single-instance AI microservice cannot handle concurrent users.	AI microservice is stateless and containerised, enabling horizontal scaling via Kubernetes. MongoDB compound indexes on {userId, sessionId} ensure query performance at scale.
8 Unauthorised Access: Users may attempt to access other users' chat sessions.	All session and message queries are scoped by the authenticated user's ID extracted from the verified token. Backend returns HTTP 403 for cross-user access attempts.

Conclusion

The Secure AI Consultation Platform has been successfully developed as a comprehensive, production-ready web application that addresses all identified pain points in healthcare information access. The project delivers a complete digital platform combining modern web development with state-of-the-art large language model technology.

The key accomplishments of this project include:

1. **Secure Authentication:** Implemented Google OAuth 2.0 with server-side token verification, ensuring that only verified users can access the platform and that session data is cryptographically tied to a verified identity.
2. **Persistent Conversation History:** Built a complete session and message management system enabling users to revisit any past conversation, providing continuity of care and a personal health query archive.
3. **AI-Powered Chat Interface:** Integrated a fine-tuned Mistral-7B LLM via a dedicated Python FastAPI microservice, with the correct Mistral Instruct prompt template, GPU-aware inference, and configurable generation parameters.
4. **Production-Ready Architecture:** The modular, containerised design with separated frontend, backend, AI service, and database layers supports independent scaling of each component and straightforward cloud deployment via Docker and AWS ECS.
5. **Responsible AI Design:** Implemented persistent healthcare disclaimers, recommended professional consultation triggers, and constrained generation parameters to minimise the risk of harmful AI-generated medical advice.
6. **Future-Ready Fine-Tuning Pipeline:** Established a LoRA adapter training infrastructure supporting QLoRA fine-tuning, knowledge graph augmentation, and evaluation of domain-specific models (BioMistral-7B, LLaMA 3.2-3B) for improved medical response quality.

The final system comprises five database models, six backend controllers, five RESTful API route groups with over 14 endpoints, five frontend page components, three reusable UI components, and a standalone Python FastAPI AI microservice — built on the MERN stack with Google OAuth 2.0 and a LoRA-adapted Mistral-7B LLM. The application demonstrates the practical application of modern web development concepts including OAuth 2.0 authentication, parameter-efficient LLM fine-tuning, RESTful microservice architecture, NoSQL database design, and responsible AI deployment patterns.

Glossary

AI Chatbot	A conversational software application that interacts with users using natural language, powered by a trained machine learning model.
LLM	Large Language Model — a transformer-based neural network trained on large text corpora for language understanding and generation.
LoRA	Low-Rank Adaptation — a parameter-efficient fine-tuning technique injecting trainable low-rank matrices into frozen pre-trained model layers.
QLoRA	Quantised LoRA — extends LoRA with 4-bit NF4 weight quantisation to reduce GPU memory requirements while preserving model quality.
PEFT	Parameter-Efficient Fine-Tuning — umbrella term for methods fine-tuning only a subset of model parameters.
FastAPI	A modern, high-performance Python web framework for building APIs with automatic OpenAPI documentation.
MERN	MongoDB, Express.js, React, Node.js — a full-stack JavaScript development stack.
OAuth 2.0	An open authorisation framework enabling secure delegated access to user resources without sharing credentials.
REST	Representational State Transfer — an architectural style for distributed hypermedia systems using standard HTTP methods.
NLP	Natural Language Processing — the branch of AI concerned with enabling computers to understand and interpret human language.
bf16	Brain Float 16 — a 16-bit floating-point format maintaining fp32 dynamic range while halving memory usage; used in mixed-precision training.
Docker	A container platform for packaging applications and their dependencies into portable, reproducible images.
Kubernetes	A container orchestration platform automating deployment, scaling, and management of containerised applications.

References

- [1] Vaswani et al., “Attention Is All You Need,” NeurIPS 2017. <https://arxiv.org/abs/1706.03762>
- [2] Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” ICLR 2022. <https://arxiv.org/pdf/2305.10435>
- [3] Dettmers et al., “QLoRA: Efficient Finetuning of Quantized LLMs.” <https://arxiv.org/pdf/2305.14314>
- [4] <https://arxiv.org/pdf/2407.21783>
- [5] <https://arxiv.org/pdf/2401.02385>
- [6] <https://arxiv.org/pdf/2107.03134>
- [7] <https://arxiv.org/pdf/2402.10373>
- [8] <https://arxiv.org/html/2510.26788v1>
- [9] “FP64, FP32, FP16, BFloat16, TF32 and other members of the zoo.” <https://moocaholic.medium.com/fp64-fp32-fp16-bfloat16-tf32-and-other-members-of-the-zoo-a1ca7897d407>
- [10] Unsloth AI — efficient LLM fine-tuning. <https://github.com/unslothai/unsloth>
- [11] Karpathy, nanoGPT benchmarks. <https://github.com/karpathy/nanoGPT/blob/master/bench.py>
- [12] MongoDB Documentation. <https://www.mongodb.com/docs/manual/>
- [13] Express.js Documentation. <https://expressjs.com/>
- [14] React Documentation. <https://react.dev/>
- [15] Google Identity Platform — OAuth 2.0 for Web. <https://developers.google.com/identity/protocols/oauth2>
- [16] HuggingFace Transformers. <https://huggingface.co/docs/transformers>
- [17] PEFT Library. <https://huggingface.co/docs/peft>
- [18] FastAPI Documentation. <https://fastapi.tiangolo.com/>
- [19] Vite — Next Generation Frontend Tooling. <https://vitejs.dev/>